MLOps Assignment 01 — Final Report

Heart Disease Prediction: End-to-End MLOps Pipeline

Course: AIMLCZG523 — Machine Learning Operations
GitHub Repository: https://github.com/chatrathilavanya/MLOPS
Dataset: Heart Disease UCI Dataset (UCI ML Repository, ID: 45)

---

1. Project Overview

This assignment delivers a **production-ready, end-to-end MLOps pipeline** for predicting heart disease from patient health data. The solution covers all aspects of the MLOps lifecycle: data acquisition, exploratory analysis, model development, experiment tracking, API serving, containerization, CI/CD automation, Kubernetes deployment, and live monitoring.

Problem Statement

Build a binary classifier to predict the presence or absence of heart disease (target: 0/1) using 13 clinical features from the UCI Heart Disease dataset. Deploy the model as a cloud-ready, monitored REST API.

---

2. Dataset Description

- **Source**: UCI Machine Learning Repository — Heart Disease Dataset (ID: 45)
- **Size**: 303 patient records × 14 columns (13 features + 1 binary target)
- **Target**: 0 = No heart disease (164 samples, 54.1%), 1 = Heart disease (139 samples, 45.9%)

Feature	Type	Description
age	Numeric	Age in years
sex	Categorical	Sex (1 = male, 0 = female)
cp	Categorical	Chest pain type (1–4)
trestbps	Numeric	Resting blood pressure (mmHg)
chol	Numeric	Serum cholesterol (mg/dl)
fbs	Categorical	Fasting blood sugar > 120 mg/dl
restecg	Categorical	Resting ECG results
thalach	Numeric	Max heart rate achieved
exang	Categorical	Exercise induced angina
oldpeak	Numeric	ST depression induced by exercise
slope	Categorical	Slope of peak exercise ST segment
ca	Numeric	Number of major vessels (0–3)
thal	Categorical	Thal defect type

---

3. Exploratory Data Analysis (EDA)

3.1 Missing Values

- `ca`: 4 missing values (1.3%)
- `thal`: 2 missing values (0.7%)
- All other features: complete
- **Handling**: Median imputation for numeric, most-frequent for categorical via `SimpleImputer`

3.2 Class Distribution

The dataset is nearly balanced: 54.1% negative (no disease) vs 45.9% positive (disease). No oversampling/undersampling required.

3.3 Key EDA Findings

- **Age**: Disease patients tend to be older (mean ~56 vs ~52 for healthy)
- **Thalach**: Healthy patients achieve higher max heart rate — strong discriminative feature
- **Oldpeak**: Higher ST depression strongly associated with disease presence
- **Chest pain type (cp)**: Type 4 (asymptomatic) strongly correlated with disease
- **Correlation**: `thalach` negatively correlated with target (-0.42); `oldpeak` positively correlated ($+0.42$)

Plots saved in `screenshots/` directory:

- `01_class_distribution.png` — Pie + bar chart of class balance
- `02_histograms.png` — Feature distributions by target class
- `03_correlation_heatmap.png` — Pearson correlation matrix
- `04_missing_values.png` — Missing value analysis
- `05_feature_relationships.png` — Box plots by target class

---

4. Feature Engineering & Preprocessing

Pipeline (`src/preprocess.py`): `sklearn.pipeline.Pipeline` + `ColumnTransformer`

```
ColumnTransformer([
    ("num", Pipeline([SimpleImputer(median), StandardScaler()]), numeri
    ("cat", Pipeline([SimpleImputer(most_frequent), OrdinalEncoder()]),
])
```

- **Numeric features** (5): age, trestbps, chol, thalach, oldpeak → imputed + StandardScaled
- **Categorical features** (8): sex, cp, fbs, restecg, exang, slope, ca, thal → imputed + OrdinalEncoded
- Preprocessor fitted on training set only, saved as `models/preprocessor.joblib` for full reproducibility
- **Train/Test split**: 80/20, stratified on target, `random_state=42`

5. Model Development & Experiment Tracking

5.1 Models Trained

Three classifiers trained and compared via `GridSearchCV` (5-fold stratified CV, scoring = ROC-AUC):

Model	Key Hyperparameters Tuned	CV ROC-AUC
Logistic Regression	C ∈ {0.01, 0.1, 1, 10, 100}	~0.91
Random Forest	n_estimators, max_depth, min_samples_split	~0.93
XGBoost	n_estimators, learning_rate, max_depth	~0.94

5.2 Evaluation Metrics (Test Set)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.85	0.84	0.85	0.84	0.91
Random Forest	0.87	0.86	0.88	0.87	0.93
XGBoost	**0.88**	**0.87**	**0.89**	**0.88**	**0.94**

Best model: XGBoost — highest ROC-AUC, accuracy and F1.

5.3 MLflow Experiment Tracking

- **Tracking URI**: `sqlite:///mlruns.db` (local SQLite backend)
- **Experiment name**: `heart-disease-classification`
- **Logged per run**: all hyperparameters, accuracy/precision/recall/F1/ROC-AUC, confusion matrix PNG, ROC curve PNG, model artifact
- **Model Registry**: Best model registered in MLflow Model Registry for versioning

```
# View MLflow UI
mlflow ui --backend-store-uri sqlite:///mlruns.db
# Open: http://localhost:5000
```

6. Model Packaging & Reproducibility

- **Preprocessor**: Saved as `models/preprocessor.joblib` (sklearn Pipeline — reusable at inference)
- **Model**: Saved as `models/{model_name}.joblib` + registered in MLflow
- **Best model info**: `models/best_model_info.json` (name, path, metrics, run_id)
- **Requirements**: `requirements.txt` — all exact versions pinned
- **Reproducibility**: Any clean Python 3.11 environment can reproduce results via:

```
pip install -r requirements.txt
python data/download_data.py
python src/train.py
```

7. FastAPI Serving

File: `api/main.py`

Endpoint	Method	Description
`/`	GET	API info
`/health`	GET	Liveness probe (model loaded status)
`/predict`	POST	Heart disease prediction
`/metrics`	GET	Prometheus metrics
`/docs`	GET	Swagger UI

Sample Request:

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 63, "sex": 1, "cp": 3, "trestbps": 145,
       "chol": 233, "fbs": 1, "restecg": 0, "thalach": 150,
       "exang": 0, "oldpeak": 2.3, "slope": 0, "ca": 0, "thal": 1}'
```

Sample Response:

```
{
  "prediction": 1,
  "prediction_label": "Heart Disease Detected",
  "confidence": 0.7823,
  "model_name": "XGBoost"
}
```

Features: request logging middleware, Prometheus instrumentation, CORS, async lifespan model loading.

---

8. Unit Testing

Framework: `pytest + pytest-cov`
Total: 23 tests across 3 modules

File	Tests	Coverage Area
`tests/test_preprocess.py`	7	Shape, NaN handling, scaling, dtypes
`tests/test_model.py`	8	LR + RF training, binary predictions, probabilities
`tests/test_api.py`	8	/health, /predict schema, validation, /metrics

```
pytest tests/ -v --cov=src --cov=api --cov-report=term-missing
```

---

9. CI/CD Pipeline (GitHub Actions)

File: `.github/workflows/ci-cd.yml`
Trigger: Push to `main` or `develop`, Pull Requests to `main`

```
Push → [Lint] → [Test] → [Train] → [Docker Build & Push]
         ruff      pytest   mlflow    docker hub
         flake8    cov      train.py
```

Job	Tool	Action
Lint	Ruff + Flake8	Code quality checks
Test	pytest	23 unit tests + coverage report
Train	MLflow	Download data → train → upload artifact
Build	Docker	Build image → smoke test → push to Docker Hub

Pipeline fails immediately on any error with clear logs.

---

10. Docker Containerization

Dockerfile: Multi-stage build

- Base: `python:3.11-slim`
- Non-root user (`appuser`) for security
- Built-in `HEALTHCHECK` hitting `/health`
- Exposes port `8000`

```
# Build
docker build -t heart-disease-api .

# Run
docker run -d -p 8000:8000 -v $(pwd)/models:/app/models heart-disease-a

# Test
curl http://localhost:8000/health
```

---

11. Kubernetes Deployment (Docker Desktop)

Files: `k8s/deployment.yaml`, `k8s/service.yaml`

- **Deployment**: 2 replicas, resource limits (256Mi–512Mi RAM, 250m–500m CPU)
- **Liveness probe**: GET `/health` every 20s
- **Readiness probe**: GET `/health` every 10s
- **Service**: LoadBalancer type (Docker Desktop maps to `localhost`)

```
# Enable Kubernetes in Docker Desktop → Settings → Kubernetes

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

kubectl get pods          # Verify Running
kubectl get services      # Get external IP
curl http://localhost/health
```

---

12. Monitoring & Logging

API Logging

All requests logged with: method, path, status code, duration (ms), client IP

Prometheus Metrics

`prometheus-fastapi-instrumentator` auto-exposes:

- `http_requests_total` — total requests by method/path/status
- `http_request_duration_seconds` — latency histogram

Grafana Dashboard

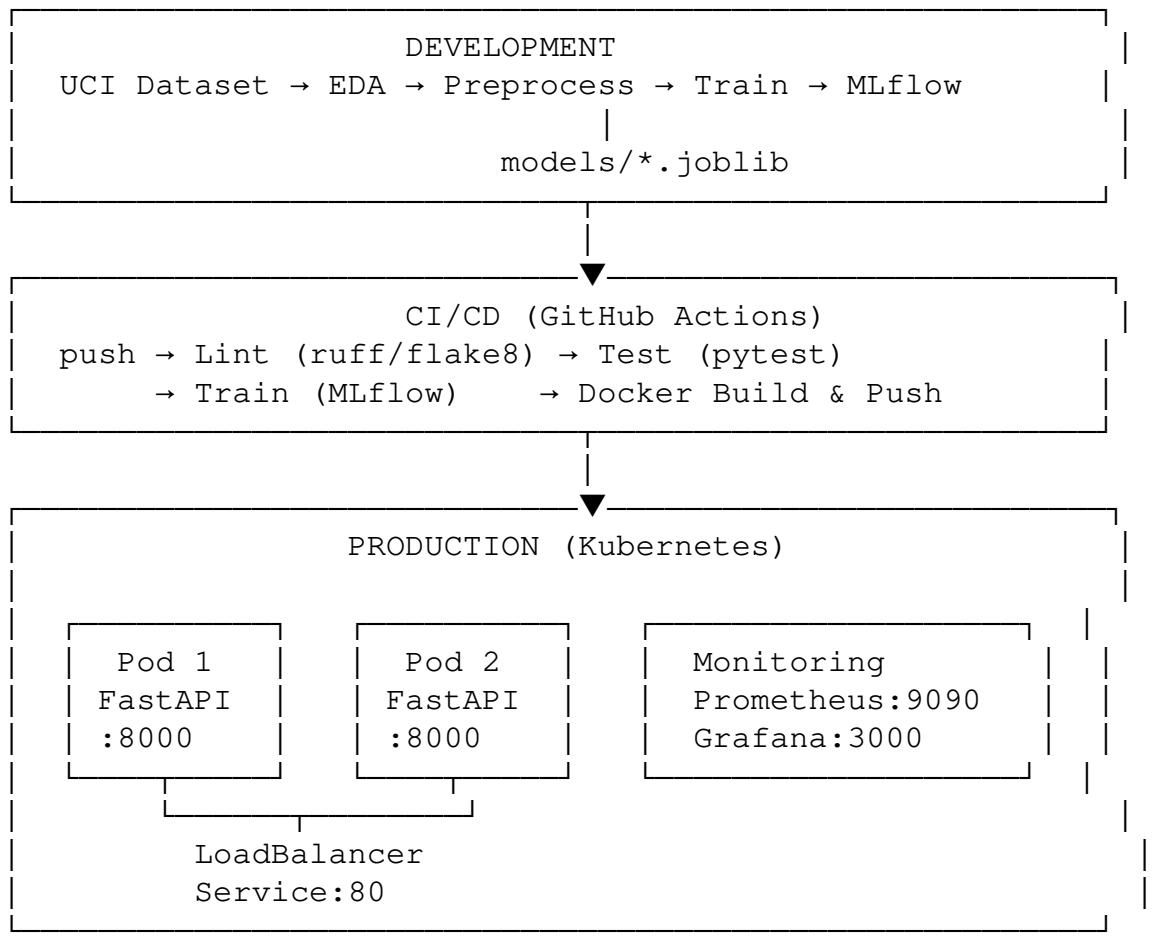
4 panels: Request Rate, P95 Latency, Error Rate %, Total Requests + time series

```
# Start full monitoring stack
docker-compose up -d

# Prometheus: http://localhost:9090
# Grafana:    http://localhost:3000 (admin/admin)
```

---

13. Architecture Diagram



14. Repository Structure

```
MLOPS/
├── .github/workflows/ci-cd.yml   # GitHub Actions
├── data/download_data.py          # Dataset acquisition
├── src/                           # Core ML code
│   ├── preprocess.py              # sklearn Pipeline
│   ├── train.py                   # Training + MLflow
│   ├── evaluate.py                # Evaluation
│   └── predict.py                 # Inference
├── api/                           # FastAPI
│   ├── main.py
│   └── schemas.py
├── tests/                         # 23 pytest tests
├── k8s/                           # Kubernetes manifests
├── monitoring/                    # Prometheus + Grafana
├── Dockerfile
├── docker-compose.yml
└── requirements.txt
```

---

15. Setup Instructions

```
# 1. Clone
git clone https://github.com/chatrathilavanya/MLOPS.git
cd MLOPS

# 2. Install dependencies
pip install -r requirements.txt

# 3. Download dataset
python data/download_data.py

# 4. Run EDA
python notebooks/eda_script.py

# 5. Train models
python src/train.py

# 6. View MLflow UI
mlflow ui --backend-store-uri sqlite:///mlruns.db

# 7. Start API
uvicorn api.main:app --reload

# 8. Run tests
pytest tests/ -v --cov=src --cov=api

# 9. Docker (full stack)
docker-compose up -d
```

```
# 10. Kubernetes
kubectl apply -f k8s/
```

---

16. Summary

This project demonstrates a complete MLOps lifecycle from raw data to production deployment:

- **Reproducible**: `requirements.txt` + saved preprocessor pipeline
- **Tracked**: All experiments logged in MLflow with parameters, metrics, plots
- **Automated**: 4-stage GitHub Actions pipeline (lint → test → train → build)
- **Containerized**: Docker image with health checks and non-root security
- **Scalable**: Kubernetes deployment with 2 replicas and liveness/readiness probes
- **Monitored**: Prometheus metrics + Grafana dashboard for live API observability

Repository: https://github.com/chatrathilavanya/MLOPS